

# Task 2: Database Integration & Migrations

Spring Boot + Spring Data JPA + Flyway | Department & Employee Schema

---

## Goal

Connect the application to a relational database and manage its schema using version-controlled **Flyway** migrations, so schema changes are repeatable, auditable, and applied automatically at application startup. This builds on the Employee domain from Task 1, adding a **Department** entity with a one-to-many relationship to **Employee**.

## Requirements Covered

- Relational DB schema designed with a Department (parent) and Employee (child) table
- Spring Data JPA entities and repositories mapped to the schema
- Schema versioning and migration with Flyway (V1, V2, V3... naming convention)
- Seed data inserted via migration scripts so the app has sample data on first run
- Configuration for both MySQL/PostgreSQL (production) and H2 (local/dev)
- Sample native and JPQL queries against the migrated schema

## Project Structure

```
com.internship.employeeapi
|-- entity/Department.java
|-- entity/Employee.java
|-- repository/DepartmentRepository.java
|-- repository/EmployeeRepository.java
resources/
|-- application.properties
|-- application-mysql.properties
|-- db/migration/
|   |-- V1__create_departments_table.sql
|   |-- V2__create_employees_table.sql
|   |-- V3__seed_departments.sql
|   |-- V4__seed_employees.sql
|   |-- V5__add_index_employees_department.sql
pom.xml (Flyway dependency added)
```

# 1. Maven Dependencies (Flyway + DB drivers)

## pom.xml (additions)

```
<!-- Add to the <dependencies> section of pom.xml from Task 1 -->

<!-- Flyway core -->
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
</dependency>

<!-- Required by Flyway for MySQL 8+ -->
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-mysql</artifactId>
</dependency>

<!-- MySQL driver (use if targeting MySQL) -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- PostgreSQL driver (use instead, if targeting PostgreSQL) -->
<!--
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
-->
```

# 2. Configuration

## src/main/resources/application.properties

```
# ===== Default profile: H2 in-memory (local/dev, Flyway still runs) =====
server.port=8080

spring.datasource.url=jdbc:h2:mem:employee;MODE=MySQL
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# IMPORTANT: let Flyway own the schema, not Hibernate.
# validate = Hibernate checks entities match the Flyway-created schema but never alters it.
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
spring.flyway.baseline-on-migrate=true

spring.h2.console.enabled=true
```

## src/main/resources/application-mysql.properties

```
# Activate with: --spring.profiles.active=mysql
spring.datasource.url=jdbc:mysql://localhost:3306/employee?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=yourpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

spring.flyway.enabled=true
```

```
spring.flyway.locations=classpath:db/migration
```

### 3. Flyway Migration Scripts

Flyway executes files in db/migration in strict version order (V1, V2, V3, ...) exactly once, tracking progress in a flyway\_schema\_history table it creates automatically.

#### src/main/resources/db/migration/V1\_\_create\_departments\_table.sql

```
-- V1__create_departments_table.sql
CREATE TABLE departments (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL UNIQUE,
  location    VARCHAR(100),
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

#### src/main/resources/db/migration/V2\_\_create\_employees\_table.sql

```
-- V2__create_employees_table.sql
CREATE TABLE employees (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(100) NOT NULL UNIQUE,
  salary      DECIMAL(10,2) NOT NULL,
  department_id BIGINT NOT NULL,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_employee_department
    FOREIGN KEY (department_id) REFERENCES departments(id)
    ON DELETE RESTRICT
);
```

#### src/main/resources/db/migration/V3\_\_seed\_departments.sql

```
-- V3__seed_departments.sql
INSERT INTO departments (name, location) VALUES
  ('Engineering', 'Bengaluru'),
  ('Product',     'Bengaluru'),
  ('Human Resources', 'Mumbai'),
  ('Finance',     'Delhi');
```

#### src/main/resources/db/migration/V4\_\_seed\_employees.sql

```
-- V4__seed_employees.sql
INSERT INTO employees (name, email, salary, department_id) VALUES
  ('Asha Rao',      'asha.rao@example.com',      65000.00, 1),
  ('Ravi Kumar',    'ravi.kumar@example.com',      72000.00, 1),
  ('Meera Nair',    'meera.nair@example.com',      58000.00, 2),
  ('Sanjay Gupta',  'sanjay.gupta@example.com',    61000.00, 3),
  ('Priya Singh',   'priya.singh@example.com',      75000.00, 4);
```

#### src/main/resources/db/migration/V5\_\_add\_index\_employees\_department.sql

```
-- V5__add_index_employees_department.sql
-- Speeds up lookups/joins filtering employees by department
CREATE INDEX idx_employees_department_id ON employees(department_id);
```

## 4. JPA Entities Mapped to the Migrated Schema

**src/main/java/com/internship/employeeapi/entity/Department.java**

```
package com.internship.employeeapi.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "departments")
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Column(length = 100)
    private String location;

    @Column(name = "created_at", insertable = false, updatable = false)
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = false)
    private List<Employee> employees = new ArrayList<>();

    public Department() {
    }

    public Department(Long id, String name, String location) {
        this.id = id;
        this.name = name;
        this.location = location;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getLocation() { return location; }
    public void setLocation(String location) { this.location = location; }

    public LocalDateTime getCreatedAt() { return createdAt; }

    public List<Employee> getEmployees() { return employees; }
    public void setEmployees(List<Employee> employees) { this.employees = employees; }
}
```

**src/main/java/com/internship/employeeapi/entity/Employee.java**

```
package com.internship.employeeapi.entity;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

@Column(nullable = false, length = 100)
private String name;

@Column(nullable = false, unique = true, length = 100)
private String email;

@Column(nullable = false, precision = 10, scale = 2)
private BigDecimal salary;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "department_id", nullable = false)
private Department department;

@Column(name = "created_at", insertable = false, updatable = false)
private LocalDateTime createdAt;

public Employee() {
}

public Employee(Long id, String name, String email, BigDecimal salary, Department department) {
    this.id = id;
    this.name = name;
    this.email = email;
    this.salary = salary;
    this.department = department;
}

public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public BigDecimal getSalary() { return salary; }
public void setSalary(BigDecimal salary) { this.salary = salary; }

public Department getDepartment() { return department; }
public void setDepartment(Department department) { this.department = department; }

public LocalDateTime getCreatedAt() { return createdAt; }
}

```

## 5. Repositories with Sample Queries

**src/main/java/com/internship/employeeapi/repository/DepartmentRepository.java**

```
package com.internship.employeeapi.repository;

import com.internship.employeeapi.entity.Department;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface DepartmentRepository extends JpaRepository<Department, Long> {
    Optional<Department> findByName(String name);
}
```

**src/main/java/com/internship/employeeapi/repository/EmployeeRepository.java**

```
package com.internship.employeeapi.repository;

import com.internship.employeeapi.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.math.BigDecimal;
import java.util.List;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Derived query: employees belonging to a given department
    List<Employee> findByDepartmentId(Long departmentId);

    // Derived query: employees earning above a threshold
    List<Employee> findBySalaryGreaterThan(BigDecimal salary);

    // JPQL: average salary per department name
    @Query("SELECT e.department.name, AVG(e.salary) FROM Employee e GROUP BY e.department.name")
    List<Object[]> findAverageSalaryByDepartment();

    // JPQL: employees in a department, ordered by salary descending
    @Query("SELECT e FROM Employee e WHERE e.department.name = :deptName ORDER BY e.salary DESC")
    List<Employee> findByDepartmentNameOrderBySalaryDesc(@Param("deptName") String deptName);

    // Native SQL: top earner overall
    @Query(value = "SELECT * FROM employees ORDER BY salary DESC LIMIT 1", nativeQuery = true)
    Employee findTopEarner();
}
```

## Sample Queries Against the Migrated Schema

### sample-queries.sql

```
-- All employees with their department name (JOIN)
SELECT e.name, e.email, e.salary, d.name AS department, d.location
FROM employees e
JOIN departments d ON e.department_id = d.id
ORDER BY d.name, e.salary DESC;

-- Average salary per department
SELECT d.name AS department, ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
JOIN departments d ON e.department_id = d.id
GROUP BY d.name;

-- Highest paid employee per department (window function)
SELECT name, department, salary
FROM (
    SELECT e.name, d.name AS department, e.salary,
           RANK() OVER (PARTITION BY d.id ORDER BY e.salary DESC) AS rnk
    FROM employees e
    JOIN departments d ON e.department_id = d.id
) ranked
WHERE rnk = 1;

-- Departments with no employees (LEFT JOIN)
SELECT d.name
FROM departments d
LEFT JOIN employees e ON e.department_id = d.id
WHERE e.id IS NULL;
```

## How Flyway Runs

- On application startup, Flyway scans `classpath:db/migration` and compares it against the `flyway_schema_history` table.
- Any V-scripts not yet applied are executed in order, inside a transaction per script.
- Because `spring.jpa.hibernate.ddl-auto=validate`, Hibernate only checks that entity mappings match the Flyway-created schema - it never generates or alters DDL itself.
- To add a schema change later, add a new file such as `V6__add_phone_to_employees.sql` - never edit an already-applied script.

## Run with MySQL profile

### run-mysql.sh

```
# Start a local MySQL instance, then run:
mvn spring-boot:run -Dspring-boot.run.profiles=mysql

# Flyway will create employeeedb (if createDatabaseIfNotExist), run V1-V5,
# and the app will start with departments/employees already seeded.
```

## Deliverables

- DB scripts: V1-V5 Flyway migration files (schema + seed data + index) shown above.
- Configuration: `application.properties` (H2/dev) and `application-mysql.properties` (MySQL/prod).
- Sample queries: JPQL/derived queries in `EmployeeRepository` plus raw SQL analytics queries.